

Monte-Carlo-Simulator

Projekt-Handbuch zum Mitverfolgen
Jede Klasse, jede Methode, jedes Attribut erklärt

Projekt	Monte-Carlo-Simulation von Aktienkursen
Sprache	Java (JDK 22+, kein Build-Schritt)
Umfang	5 Klassen und 1 Interface, rund 400 Zeilen
Modelle	Geometrische Brownsche Bewegung, Merton-Sprungdiffusion
Ablageort	/Users/phi/VSCode/monteCarloSimulation/
Stand	4. August 2026

Inhaltsverzeichnis

Teil A - Überblick und Grundbegriffe	4
Was das Programm macht	4
Was Monte Carlo bedeutet	4
Das Grundmodell: Geometrische Brownsche Bewegung	4
Warum im Logarithmus gerechnet wird	4
Die Korrektur um sigma-Quadrat halbe	5
Das zweite Modell: Sprünge	5
Die Kompensation der Sprünge	5
Die Kennzahlen	6
VaR und CVaR	6
Maximaler Rückgang (Max Drawdown)	6
Zufall und Reproduzierbarkeit	6
Wie dieses Handbuch zu lesen ist	6
Die Dateien im Überblick	7
Teil B - Die Modellschicht	8
PriceModel.java	8
TRADING_DAYS	8
sampleLogReturn()	8
describe()	8
GbmModel.java	8
Komponenten	8
sampleLogReturn()	9
describe()	9
JumpDiffusionModel.java	9
Komponenten	9
sampleLogReturn()	9
samplePoisson()	10
describe()	10
Teil C - Der Simulationskern	11
Simulator.java	11
PathResult	11
Felder und Konstruktor	11

run()	11
samplePaths()	12

Teil D - Die Statistik 13

Stats.java	13
Felder	13
Stats()	13
mean(), std(), min(), max(), median()	13
percentile()	13
probBelow()	14
tailMean()	14
histogram()	14

Teil E - Das Hauptprogramm 16

MonteCarloSim.java	16
main()	16
exportPaths()	16
parseArgs()	17
getD()	17
Die Optionen im Überblick	17

Teil F - Ergebnisse lesen und prüfen 19

Die Probe aufs Exempel	19
Beide Modelle nebeneinander	19
Grenzen des Modells	19
Aufrufe zum Ausprobieren	20

Teil A - Überblick und Grundbegriffe

Was das Programm macht

Das Programm beantwortet eine Frage, die sich mit einer einzelnen Rechnung nicht beantworten lässt: **Welche Kursverläufe sind in einem Jahr überhaupt plausibel - und wie wahrscheinlich ist jeder davon?**

Dazu würfelt es einen Kursverlauf viele tausend Mal durch. Jeder Durchlauf ist ein möglicher Verlauf (ein *Pfad*). Aus der Gesamtheit aller Pfade entsteht eine Verteilung, aus der sich Wahrscheinlichkeiten ablesen lassen: Wie wahrscheinlich ist ein Verlust? Wie schlimm wird es im schlechtesten Prozent der Fälle? Wie tief geht es zwischendurch bergab, selbst wenn es am Ende gut ausgeht?

```
Terminal price distribution
  mean 108.42   median 106.26   stdev 21.95
  p5 76.43    p25 92.82     p75 121.59   p95 147.79

Horizon return
  mean +8.42%   median +6.26%   P(loss) 38.1%
  VaR 95% -23.57%   VaR 99% -33.32%
  CVaR 95% -29.40%   CVaR 99% -37.41%

Max drawdown along path
  median 17.6%   p75 23.0%   p95 32.4%   worst 57.8%
```

Das ist die Ausgabe eines Standardlaufs: 100.000 Pfade über ein Jahr, Startkurs 100, 8 Prozent erwartete Rendite, 20 Prozent Volatilität.

Was Monte Carlo bedeutet

Der Name stammt vom Kasino in Monaco. Die Idee: Statt eine Formel für das Ergebnis zu suchen, simuliert man den Zufallsprozess sehr oft und zählt aus, was dabei herauskommt.

Für die einfachsten Fragen gäbe es geschlossene Formeln - der Erwartungswert des Endkurses etwa lässt sich direkt ausrechnen. Für andere Fragen gibt es keine handliche Formel, und genau dort spielt Monte Carlo seine Stärke aus. Der **maximale Rückgang innerhalb des Pfades** ist so ein Fall: Er hängt vom gesamten Verlauf ab, nicht nur vom Endpunkt. Simulieren ist hier der einzige praktikable Weg.

Monte-Carlo-Ergebnisse sind selbst zufällig. Verdoppelt man die Pfadzahl, halbiert sich der Fehler nicht - er sinkt nur mit der Wurzel. Für vier Mal genauere Werte braucht man sechzehn Mal so viele Pfade. Deshalb ist der Standardwert mit 100.000 Pfaden bewusst hoch angesetzt; er kostet trotzdem nur rund 90 Millisekunden.

Das Grundmodell: Geometrische Brownsche Bewegung

Das Standardmodell für Aktienkurse heißt **geometrische Brownsche Bewegung** (GBM). Es beschreibt den Kurs über zwei Zahlen:

- **Drift** (μ) - die erwartete Rendite pro Jahr, also der Trend.
- **Volatilität** (σ) - wie stark der Kurs um diesen Trend schwankt, ebenfalls auf ein Jahr bezogen.

Entscheidend ist das Wort *geometrisch*: Nicht der Kurs selbst schwankt um einen festen Betrag, sondern seine **prozentuale Veränderung**. Eine Aktie bei 500 Euro bewegt sich in Euro stärker als eine bei 5 Euro - in Prozent aber gleich viel. Genau das bildet das Modell ab, und daraus folgt nebenbei, dass der Kurs nie negativ werden kann.

Warum im Logarithmus gerechnet wird

Der Code führt nie den Kurs selbst mit, sondern immer dessen Logarithmus:

```
double logPrice = Math.log(s0);
for (int d = 0; d < days; d++) {
    logPrice += model.sampleLogReturn(rng, dt);
}
// erst ganz am Ende zurückrechnen:
Math.exp(logPrice)
```

Das hat drei Gründe. Erstens werden Renditen dadurch **addiert** statt multipliziert - eine Addition ist schneller und numerisch stabiler als tausendfaches Multiplizieren. Zweitens kann der Kurs so nie negativ werden, denn `exp` liefert immer einen positiven Wert. Drittens sind die Zufallszahlen im Logarithmus normalverteilt, und Normalverteilungen sind das, was der Zufallsgenerator direkt liefert.

Die Korrektur um sigma-Quadrat halbe

Im Code steht an zentraler Stelle nicht einfach `mu`, sondern:

```
(mu - 0.5 * sigma * sigma) * dt + sigma * Math.sqrt(dt) * rng.nextGaussian()
```

Der Abzug von `0.5 * sigma * sigma` ist kein Schönheitsfehler, sondern zwingend. Der Grund: Der **Durchschnitt** der Kurse und der **typische** Kurs sind bei einer schiefen Verteilung nicht dasselbe. Ein Verlust von 50 Prozent wird durch einen Gewinn von 50 Prozent nicht ausgeglichen - man landet bei 75 statt bei 100. Schwankung frisst also Rendite, und zwar genau um **sigma-Quadrat halbe** pro Jahr.

Ohne diesen Abzug wäre die simulierte Durchschnittsrendite systematisch zu hoch. Mit ihm gilt bei den Standardwerten (`mu = 8 Prozent`, `sigma = 20 Prozent`):

```
Abzug:          0.20^2 / 2 = 0.02 = 2 %/Jahr
Drift im Log:    0.08 - 0.02 = 0.06

Erwarteter Mittelwert: 100 * e^0.08 = 108.33
Erwarteter Median:    100 * e^0.06 = 106.18
```

Der Simulator lieferte im Test 108.42 und 106.26 - beides innerhalb der zu erwartenden Monte-Carlo-Streuung. Der Unterschied zwischen beiden Zahlen ist keine Ungenauigkeit, sondern echte Eigenschaft der Verteilung: Der Mittelwert wird von wenigen sehr guten Pfaden nach oben gezogen, während die Mehrheit der Pfade unter ihm liegt.

Das zweite Modell: Sprünge

Die GBM hat eine bekannte Schwäche: Sie erzeugt zu wenige Extremtage. In echten Kursreihen kommen Tagesverluste von 7 oder 10 Prozent deutlich häufiger vor, als eine Normalverteilung zulässt.

Das **Merton-Sprungdiffusionsmodell** ergänzt die GBM deshalb um seltene, plötzliche Sprünge. Es kombiniert zwei Zufallsquellen:

- die gewohnte stetige Schwankung (die *Diffusion*),
- plus gelegentliche Sprünge, deren Anzahl einem **Poisson-Prozess** mit der Rate `lambda` folgt. Jeder Sprung verändert den Kurs um einen zufälligen Faktor.

Bei den Standardwerten (`lambda = 4`, mittlere Sprunghöhe -3 Prozent) passieren im Schnitt vier Sprünge pro Jahr, überwiegend nach unten - das Modell bildet also Schockereignisse ab.

Die Kompensation der Sprünge

Wenn Sprünge im Mittel nach unten gehen, würden sie die Gesamtrendite drücken - μ würde seine Bedeutung verlieren. Deshalb wird der Drift der Diffusion um den erwarteten Sprungbeitrag **angehoben**:

```
double kappa = Math.exp(m1(jumpMean + 0.5 * jumpStd * jumpStd)); //  $E[e^J] - 1$ 
double diffusion = (mu - lambda * kappa - 0.5 * sigma * sigma) * dt + ...
```

κ ist die mittlere prozentuale Kursänderung je Sprung. Bei den Standardwerten ergibt sich etwa -2,8 Prozent je Sprung und damit rund -11,3 Prozent pro Jahr, die der Diffusionsdrift wieder ausgleicht. Dadurch bleibt μ in **beiden** Modellen dasselbe: die erwartete Gesamtrendite.

Genau das bestätigt der Test: Mit denselben Vorgaben lieferte die GBM einen Mittelwert von 108.42, das Sprungmodell 108.29 - praktisch identisch. Was sich ändert, sind ausschließlich die Ränder der Verteilung: CVaR 99 Prozent verschlechtert sich von -37 auf -45 Prozent, der schlimmste Rückgang von 58 auf 71 Prozent. Sprünge verschieben also nicht den Erwartungswert, sondern das Risiko.

Die Kennzahlen

VaR und CVaR

Beide beschreiben den linken Rand der Verteilung, also den Verlustfall:

- **VaR 95 Prozent** (Value at Risk) ist die Rendite am 5-Prozent-Perzentil. Aussage: In 95 von 100 Fällen fällt das Ergebnis besser aus als dieser Wert.
- **CVaR 95 Prozent** (auch *Expected Shortfall*) ist der **Durchschnitt** aller Ergebnisse unterhalb dieser Schwelle. Aussage: Wenn es schlecht läuft, wie schlecht dann im Mittel?

Der CVaR ist die ehrlichere Zahl. Der VaR sagt nur, wo die Grenze liegt, aber nichts darüber, wie weit es dahinter noch nach unten geht. Zwei Verteilungen können denselben VaR haben und völlig unterschiedliche Katastrophenfälle - der CVaR unterscheidet sie.

Maximaler Rückgang (Max Drawdown)

Der tiefste Rückgang von einem zwischenzeitlichen Höchststand bis zum darauffolgenden Tief, gemessen entlang des Pfades. Diese Zahl ist für die Praxis oft wichtiger als das Endergebnis: Ein Pfad, der am Ende bei plus 8 Prozent landet, zwischendurch aber 40 Prozent im Minus lag, ist kaum durchzuhalten - und bei einem Konto mit Verlustgrenze sogar tödlich.

Der Median liegt bei 17,6 Prozent. Das heißt: Selbst im typischen Verlauf einer Aktie mit ordentlicher Durchschnittsrendite gibt es unterwegs einen Rückgang von gut 17 Prozent. Diese Zahl bekommt man aus keiner geschlossenen Formel - sie ist das eigentliche Argument für die Simulation.

Zufall und Reproduzierbarkeit

Computer würfeln nicht wirklich. Sie erzeugen aus einem Startwert (dem **Seed**) eine Folge, die zufällig *aussieht*, aber vollständig festgelegt ist. Derselbe Seed liefert deshalb immer dasselbe Ergebnis - hier erwünscht: Ein Lauf lässt sich exakt wiederholen und überprüfen. Über `--seed` kann man bewusst einen anderen Zufallsverlauf wählen.

Schwierig wird das, sobald die Pfade **parallel** auf mehreren Prozessorkernen laufen: Die Reihenfolge, in der die Kerne fertig werden, ist nicht vorhersagbar. Zögen alle Pfade aus einem gemeinsamen Generator, hänge das Ergebnis von dieser Reihenfolge ab und wäre nicht mehr reproduzierbar. Wie der Code dieses Problem löst, steht in Teil C bei `run()`.

Wie dieses Handbuch zu lesen ist

Teil B beschreibt die Modellschicht, Teil C den Simulationskern, Teil D die Statistik und Teil E das Hauptprogramm. Teil F zeigt, wie die Ergebnisse zu lesen sind und wo die Grenzen des Modells liegen. Jeder Eintrag folgt demselben Aufbau:

- **Signatur** im blau umrandeten Kasten - so steht es im Quelltext.
- **Zweck** - was die Methode in einem Satz leistet.
- **Parameter und Rückgabe** - als Tabelle, wenn vorhanden.
- **Ablauf** - die Schritte in der Reihenfolge, in der sie im Code stehen.
- **Hinweis** im gelben Kasten - Fallstricke und Designentscheidungen.

Die Dateien im Überblick

Datei	Art	Aufgabe
PriceModel.java	Interface	Gemeinsamer Vertrag aller Kursmodelle
GbmModel.java	Record	Geometrische Brownsche Bewegung
JumpDiffusionModel.java	Record	Merton-Sprungdiffusion
Simulator.java	Klasse	Erzeugt die Pfade, parallel und reproduzierbar
Stats.java	Klasse	Perzentile, VaR/CVaR, Histogramm
MonteCarloSim.java	Klasse	Kommandozeile, Ausgabe, CSV-Export

Die Aufteilung folgt einer klaren Linie: **Was** gewürfelt wird, steht in der Modellschicht. **Wie oft und wie** gewürfelt wird, steht im Simulator. **Was aus den Ergebnissen gelesen** wird, steht in Stats. Und **wie das Ganze bedient** wird, steht in MonteCarloSim. Ein neues Modell hinzuzufügen bedeutet deshalb: eine Datei schreiben, eine Zeile im Auswahlhalter ergänzen - sonst nichts.

Teil B - Die Modellschicht

PriceModel.java

Ein Interface mit zwei Methoden und einer Konstanten - der gemeinsame Vertrag, den jedes Kursmodell erfüllen muss. Der Simulator kennt **nur** dieses Interface und nicht die konkreten Modelle. Genau deshalb muss er nicht angefasst werden, wenn ein neues Modell dazukommt.

TRADING_DAYS

```
int TRADING_DAYS = 252;
```

Die Anzahl der Börsentage pro Jahr. Sie übersetzt zwischen der Jahresskala, in der μ und σ angegeben werden, und der Tagesskala, in der simuliert wird: Ein Zeitschritt ist $1.0 / 252$ Jahre lang.

In einem Interface ist jedes Feld automatisch `public static final`, auch ohne dass es dasteht. Die Konstante gehört bewusst hierher und nicht in den Simulator, weil sie die Zeitskala der Modelle definiert - sie legt fest, was `dt` überhaupt bedeutet.

sampleLogReturn()

```
double sampleLogReturn(RandomGenerator rng, double dt);
```

Zweck: Die eine Methode, um die sich alles dreht. Sie zieht **eine** zufällige Log-Rendite für einen Zeitschritt der Länge `dt`.

Parameter	Bedeutung
<code>rng</code>	Der Zufallsgenerator, aus dem gezogen wird - wird von außen übergeben
<code>dt</code>	Länge des Zeitschritts in Jahren, hier also $1/252$

Dass der Generator als Parameter hereinkommt und nicht im Modell gespeichert wird, ist der Grund, warum die Modelle gefahrlos parallel benutzt werden können: Jeder Pfad bringt seinen eigenen Generator mit, das Modellobjekt selbst bleibt zustandslos und wird von allen Kernen gleichzeitig verwendet.

describe()

```
String describe();
```

Zweck: Eine lesbare Beschreibung des Modells samt seiner Parameter, die im Kopf der Ausgabe erscheint. Damit steht in jedem Protokoll, mit welchen Annahmen gerechnet wurde - ohne diese Zeile wäre ein gespeichertes Ergebnis später nicht mehr einzuordnen.

GbmModel.java

```
public record GbmModel(double mu, double sigma) implements PriceModel
```

Das Grundmodell, als `record` geschrieben. Aus dieser einen Zeile erzeugt der Compiler den Konstruktor, die Zugriffsmethoden `mu()` und `sigma()` sowie `equals`, `hashCode` und `toString`. Records sind **unveränderlich** - genau richtig für Modellparameter, die während eines Laufs feststehen.

Komponenten

Komponente	Bedeutung
mu	Erwartete Jahresrendite als Dezimalzahl (0.08 = 8 Prozent)
sigma	Jahresvolatilität als Dezimalzahl (0.20 = 20 Prozent)

sampleLogReturn()

```
public double sampleLogReturn(RandomGenerator rng, double dt)
```

```
return (mu - 0.5 * sigma * sigma) * dt + sigma * Math.sqrt(dt) * rng.nextGaussian();
```

Die gesamte Modelllogik in einer Zeile, aufgebaut aus zwei Summanden:

- **Der Driftteil** $(\mu - 0.5 \cdot \sigma \cdot \sigma) \cdot dt$ - der vorhersagbare Anteil, einschließlich der in Teil A erklärten Korrektur.
- **Der Zufallsteil** $\sigma \cdot \sqrt{dt} \cdot \text{nextGaussian}()$ - `nextGaussian()` liefert eine standardnormalverteilte Zahl (Mittelwert 0, Streuung 1), die auf die gewünschte Größe skaliert wird.

Beim Zufallsteil steht die **Wurzel** von dt , beim Driftteil dt selbst. Das ist kein Tippfehler, sondern der Kern der Brownschen Bewegung: Der Trend wächst linear mit der Zeit, die Schwankung nur mit der Wurzel. Über vier Tage ist die erwartete Bewegung viermal so groß, die Unsicherheit aber nur doppelt so groß. Genau deshalb wird ein Trend über lange Zeiträume überhaupt erst erkennbar.

describe()

```
public String describe()
```

Formatiert Modellname und Parameter in Prozent, zum Beispiel *GBM (mu=8.00%/yr, sigma=20.00%/yr)*.

JumpDiffusionModel.java

```
public record JumpDiffusionModel(double mu, double sigma, double lambda,
                                double jumpMean, double jumpStd)
    implements PriceModel
```

Das erweiterte Modell. Weil beide Modelle dasselbe Interface erfüllen, kann der Simulator ohne jede Änderung mit ihm arbeiten.

Komponenten

Komponente	Standard	Bedeutung
mu	0.08	Erwartete Jahresrendite - einschließlich der Sprünge
sigma	0.20	Volatilität des stetigen Anteils
lambda	4.0	Erwartete Anzahl Sprünge pro Jahr
jumpMean	-0.03	Mittlere Sprunghöhe im Logarithmus (negativ = Absturz)
jumpStd	0.05	Streuung der Sprunghöhe

sampleLogReturn()

```
public double sampleLogReturn(RandomGenerator rng, double dt)
```

```
double kappa = Math.expm1(jumpMean + 0.5 * jumpStd * jumpStd); //  $E[e^J] - 1$ 
double diffusion = (mu - lambda * kappa - 0.5 * sigma * sigma) * dt
    + sigma * Math.sqrt(dt) * rng.nextGaussian();

int jumps = samplePoisson(rng, lambda * dt);
double jumpSum = 0.0;
for (int i = 0; i < jumps; i++) {
    jumpSum += jumpMean + jumpStd * rng.nextGaussian();
}
return diffusion + jumpSum;
```

Ablauf:

1. kappa berechnen - die mittlere prozentuale Kursänderung je Sprung.
2. Den Diffusionsteil bilden, dessen Drift um $\lambda * \kappa$ angehoben ist, damit μ die Gesamtrendite bleibt.
3. Auswürfeln, **wie viele** Sprünge in diesem Zeitschritt stattfinden.
4. Für jeden Sprung eine Höhe ziehen und aufaddieren.
5. Beide Teile summieren - im Logarithmus bedeutet Addition eine Multiplikation der Kurse.

`Math.expm1(x)` rechnet $e^x - 1$, aber genauer als die direkte Schreibweise. Bei kleinem x liegt e^x sehr nahe bei 1, und beim Abziehen der 1 gehen führende Stellen verloren - ein klassischer Fall von Auslöschung. `expm1` umgeht das. Hier ist x etwa -0.028, also genau der kritische Bereich.

samplePoisson()

```
private static int samplePoisson(RandomGenerator rng, double mean)
```

Zweck: Zieht eine poissonverteilte Zufallszahl, also die Anzahl der Sprünge in diesem Zeitschritt. Die Poisson-Verteilung ist die passende Verteilung für seltene, unabhängige Ereignisse mit bekannter Durchschnittsrate.

```
double l = Math.exp(-mean);
int k = 0;
double p = 1.0;
do {
    k++;
    p *= rng.nextDouble();
} while (p > l);
return k - 1;
```

Das ist der Algorithmus von Knuth: Man multipliziert gleichverteilte Zufallszahlen, bis das Produkt unter die Schwelle $e^{-\text{mean}}$ fällt, und zählt die Schritte. Die Zahl der Durchläufe entspricht im Mittel $\text{mean} + 1$.

Der Algorithmus ist nur bei kleinem Mittelwert effizient - hier ist er das mit Sicherheit: $\lambda * dt$ ist 4 geteilt durch 252, also rund 0,016. In etwa 98 von 100 Zeitschritten kommt schlicht null heraus, und die Schleife läuft genau einmal. Für große Mittelwerte wäre ein anderes Verfahren nötig, das hier aber nie auftritt.

describe()

```
public String describe()
```

Gibt alle fünf Parameter lesbar aus, damit die Sprungannahmen im Protokoll nachvollziehbar bleiben.

Teil C - Der Simulationskern

Simulator.java

Diese Klasse erzeugt die Pfade. Sie kennt kein einziges Modell im Detail, sondern nur das Interface `PriceModel` - und genau deshalb funktioniert sie mit jedem Modell, das je hinzukommt.

PathResult

```
public record PathResult(double terminalPrice, double maxDrawdown) {}
```

Das Ergebnis eines einzelnen Pfades, als verschachtelter record. Statt zwei getrennter Arrays zurückzugeben, liefert der Simulator ein Array dieser Paare - so gehören Endkurs und Rückgang sichtbar zusammen und können nicht versehentlich falsch zugeordnet werden.

Komponente	Bedeutung
<code>terminalPrice</code>	Der Kurs am Ende des Zeitraums
<code>maxDrawdown</code>	Der tiefste Rückgang entlang des Pfades, als Anteil ($0.25 = 25$ Prozent)

Bewusst wird **nicht** der komplette Pfad gespeichert. Bei 100.000 Pfaden mal 252 Tagen wären das 25 Millionen Zahlen, also rund 200 MB. Beide interessierenden Kennzahlen lassen sich aber unterwegs mitführen, sodass pro Pfad nur zwei Werte übrig bleiben. Wer echte Verläufe braucht, nutzt `samplePaths()` für eine Handvoll.

Felder und Konstruktor

```
private final PriceModel model;
private final long seed;

public Simulator(PriceModel model, long seed)
```

Beide Felder sind `final`: Ein Simulator wird mit Modell und Seed angelegt und ändert sich danach nicht mehr. Das macht ihn sicher für den parallelen Zugriff - es gibt keinen veränderlichen Zustand, um den sich mehrere Kerne streiten könnten.

run()

```
public PathResult[] run(double s0, int days, int numPaths)
```

Zweck: Die Hauptmethode. Simuliert `numPaths` Pfade über `days` Tage ab dem Startkurs `s0` und liefert für jeden Endkurs und maximalen Rückgang.

```
var factory = RandomGeneratorFactory
    .<RandomGenerator.SplittableGenerator>of("L64X128MixRandom");
RandomGenerator.SplittableGenerator root = factory.create(seed);
// Pro Pfad einen eigenen Strom abspalten - in fester Reihenfolge.
RandomGenerator[] rngs = new RandomGenerator[numPaths];
for (int i = 0; i < numPaths; i++) {
    rngs[i] = root.split();
}
```

Diese Schleife ist die Antwort auf das Reproduzierbarkeitsproblem aus Teil A. Der Generator `L64X128MixRandom` ist **abspaltbar**: `split()` erzeugt einen neuen, unabhängigen Zufallsstrom. Die Ströme werden **vor** der Parallelisierung in einer festen Reihenfolge erzeugt.

Dadurch bekommt Pfad Nummer 7 immer denselben Zufallsstrom - unabhängig davon, welcher Kern ihn rechnet und wann. Das Ergebnis hängt nur vom Seed ab, nicht von der Prozessorauslastung oder der Anzahl der Kerne.

```
IntStream.range(0, numPaths).parallel().forEach(p -> {
    RandomGenerator rng = rngs[p];
    double logPrice = Math.log(s0);
    double peakLog = logPrice;
    double maxDd = 0.0;
    for (int d = 0; d < days; d++) {
        logPrice += model.sampleLogReturn(rng, dt);
        if (logPrice > peakLog) {
            peakLog = logPrice;
        } else {
            double dd = 1.0 - Math.exp(logPrice - peakLog);
            if (dd > maxDd) maxDd = dd;
        }
    }
    results[p] = new PathResult(Math.exp(logPrice), maxDd);
});
```

Ablauf je Pfad:

1. Im Logarithmus beim Startkurs beginnen; der Startwert ist zugleich der erste Höchststand.
2. Für jeden Tag eine Log-Rendite ziehen und aufaddieren.
3. Liegt der neue Wert über dem bisherigen Höchststand, wird dieser aktualisiert.
4. Sonst den aktuellen Rückgang gegenüber dem Höchststand berechnen und das Maximum merken.
5. Am Ende einmal zurück in die Kursskala rechnen und das Ergebnis ablegen.

Die Rückgangsformel $1 - \exp(\logPrice - peakLog)$ wirkt sperrig, ist aber genau der Anteil $(Höchstkurs - Kurs) / Höchstkurs$. Da die Differenz zweier Logarithmen dem Verhältnis der Kurse entspricht, genügt eine einzige exp-Berechnung statt zwei Rückrechnungen pro Tag.

`IntStream.parallel()` verteilt die Pfade automatisch auf alle Prozessorkerne. Das ist hier gefahrlos, weil jeder Pfad in eine **eigene** Position des Ergebnis-Arrays schreibt - zwei Kerne fassen nie dieselbe Stelle an. Wäre dagegen eine gemeinsame Summe fortgeschrieben worden, bräuchte es Synchronisierung.

samplePaths()

```
public double[][] samplePaths(double s0, int days, int numPaths)
```

Zweck: Liefert einige wenige **vollständige** Verläufe mit dem Kurs an jedem Tag - für den CSV-Export und damit für Diagramme.

Rückgabe: ein zweidimensionales Array `[Pfad][Tag]` mit `days + 1` Spalten. Das Plus eins ist der Startkurs, der in Spalte 0 steht.

Die Methode arbeitet bewusst **sequenziell** und spaltet die Ströme in derselben Reihenfolge ab wie `run()`. Damit ist Pfad 3 hier exakt derselbe Verlauf, der in `run()` als Pfad 3 gerechnet wurde - der exportierte Verlauf gehört also wirklich zu den ausgewiesenen Kennzahlen und ist kein unabhängiger neuer Wurf.

Weil hier jeder Tageskurs gespeichert wird, ist die Methode nur für kleine Pfadzahlen gedacht; `exportPaths` ruft sie mit 20 auf. Mit 100.000 Pfaden würde sie den Speicher sprengen - genau der Grund, warum `run()` nur zwei Kennzahlen behält.

Teil D - Die Statistik

Stats.java

Wertet eine Stichprobe aus. Im Hauptprogramm wird die Klasse dreimal verwendet: für die Endkurse, für die Renditen und für die Rückgänge. Alle Kennzahlen der Ausgabe stammen von hier.

Felder

Feld	Typ	Bedeutung
sorted	private final double[]	Eine sortierte Kopie der Stichprobe
mean	private final double	Arithmetischer Mittelwert
std	private final double	Standardabweichung

Stats()

```
public Stats(double[] values)
```

Zweck: Berechnet Mittelwert und Streuung und legt eine sortierte Kopie an.

```
this.sorted = values.clone();
Arrays.sort(this.sorted);
...
this.std = Math.sqrt(sq / (values.length - 1));
```

Zwei Details verdienen Beachtung. Erstens das `clone()`: Ohne die Kopie würde `Arrays.sort` das **übergebene** Array umsortieren. Im Hauptprogramm würden dadurch die Endkurse in eine andere Reihenfolge geraten als die Rückgänge, und die Zuordnung Pfad zu Ergebnis wäre zerstört. Zweitens das `length - 1` im Nenner - die Bessel-Korrektur, die aus einer Stichprobe die Streuung der Grundgesamtheit schätzt.

Einmal sortieren zu Beginn ist zugleich die Grundlage aller weiteren Methoden: Perzentile, Minimum, Maximum, VaR und CVaR sind auf einem sortierten Array jeweils nur noch ein Zugriff oder eine kurze Teilsumme.

mean(), std(), min(), max(), median()

```
public double mean()
public double std()
public double min()
public double max()
public double median()
```

Die einfachen Abfragen. `mean` und `std` geben die im Konstruktor berechneten Werte zurück. `min` und `max` sind das erste und letzte Element des sortierten Arrays. `median` ist der Wert bei 50 Prozent und ruft schlicht `percentile(50)` auf.

percentile()

```
public double percentile(double p)
```

Zweck: Der Wert, unterhalb dessen `p` Prozent der Stichprobe liegen. Das Arbeitspferd der Klasse.

```
double idx = p / 100.0 * (sorted.length - 1);
int lo = (int) Math.floor(idx);
int hi = (int) Math.ceil(idx);
double frac = idx - lo;
return sorted[lo] * (1 - frac) + sorted[hi] * frac;
```

Die gesuchte Position liegt meist **zwischen** zwei Elementen. Statt zu runden, wird linear zwischen den beiden Nachbarn interpoliert. Bei 100.000 Werten macht das kaum einen Unterschied, bei kleinen Stichproben aber sehr wohl - und es vermeidet Sprünge, wenn sich die Stichprobengröße leicht ändert.

probBelow()

```
public double probBelow(double threshold)
```

Zweck: Der Anteil der Stichprobe unterhalb einer Schwelle. Liefert die Verlustwahrscheinlichkeit (probBelow(0.0) auf den Renditen) und, als Gegenwert, die Wahrscheinlichkeit für das Erreichen eines Kursziels.

```
int i = Arrays.binarySearch(sorted, threshold);
int insertion = i >= 0 ? i : -(i + 1);
return (double) insertion / sorted.length;
```

Arrays.binarySearch liefert bei einem Treffer den Index, sonst einen **negativen** Wert, aus dem sich die Einfügeposition mit $-(i + 1)$ zurückrechnen lässt. Diese Position ist genau die Anzahl der Elemente unterhalb der Schwelle. Die binäre Suche braucht dafür nur rund 17 Vergleiche statt 100.000.

Bei exakten Treffern gibt binarySearch *irgendeinen* passenden Index zurück, nicht zwingend den ersten. Bei mehrfach vorkommenden Werten wäre das Ergebnis deshalb minimal ungenau. Für stetige Simulationsergebnisse ist ein exakter Treffer praktisch ausgeschlossen, sodass der Fall hier nicht auftritt.

tailMean()

```
public double tailMean(double p)
```

Zweck: Der Mittelwert des schlechtesten p-Prozent-Anteils - also der CVaR beziehungsweise Expected Shortfall.

```
int n = Math.max(1, (int) Math.floor(p / 100.0 * sorted.length));
double sum = 0;
for (int i = 0; i < n; i++) sum += sorted[i];
return sum / n;
```

Weil das Array sortiert ist, stehen die schlechtesten Werte am Anfang - es genügt, die ersten n Einträge zu mitteln. Das Math.max(1, ...) verhindert eine Division durch null, wenn der Anteil so klein ist, dass rechnerisch kein einziges Element übrig bliebe.

histogram()

```
public String histogram(int bins, int width)
```

Zweck: Erzeugt ein Balkendiagramm aus Textzeichen, das die Form der Verteilung direkt im Terminal sichtbar macht.

Parameter	Bedeutung
bins	Anzahl der Klassen, also Zeilen im Diagramm (Aufruf: 20)

Parameter	Bedeutung
width	Breite des längsten Balkens in Zeichen (Aufruf: 44)

Ablauf:

1. Die Ränder werden auf das 0,5- und 99,5-Prozent-Perzentil gesetzt statt auf Minimum und Maximum.
2. Ist die Spannweite null, wird abgebrochen - sonst käme es zur Division durch null.
3. Jeder Wert wird seiner Klasse zugeordnet und gezählt; Werte außerhalb der Ränder fallen durch die Bereichsprüfung heraus.
4. Die Balken werden auf die häufigste Klasse skaliert, sodass der längste genau `width` Zeichen misst.

Das Abschneiden der äußersten Ränder ist Absicht. Einzelne Extrempfade würden die Skala so weit dehnen, dass die eigentliche Verteilung zu ein paar Zeilen zusammenschrumpft. Die Extremwerte gehen dabei nicht verloren - sie stehen als Minimum und Maximum in der Zeile darüber.

Teil E - Das Hauptprogramm

MonteCarloSim.java

Die Klasse, die alles zusammenführt: Kommandozeile lesen, Modell bauen, Simulation starten, Kennzahlen ausgeben, optional exportieren.

main()

```
public static void main(String[] args)
```

Ablauf:

1. Argumente einlesen und die Werte mit Standardvorgaben herausziehen.
2. Das Modell über einen `switch` erzeugen - hier entscheidet sich `gbm` gegen `jump`.
3. Simulator anlegen, Laufzeit messen und die Pfade rechnen.
4. Die Ergebnisse in drei Arrays aufteilen: Endkurse, Renditen und Rückgänge.
5. Aus jedem Array ein `Stats`-Objekt bauen.
6. Die Kennzahlen blockweise ausgeben, dann das Histogramm.
7. Wenn `--export` angegeben wurde, die Beispielpfade schreiben.

```
PriceModel model = switch (modelName) {
    case "gbm" -> new GbmModel(mu, sigma);
    case "jump" -> new JumpDiffusionModel(mu, sigma,
        getD(opts, "lambda", 4.0),
        getD(opts, "jump-mean", -0.03),
        getD(opts, "jump-std", 0.05));
    default -> throw new IllegalArgumentException(
        "Unknown model '" + modelName + "' (use gbm or jump)");
};
```

Dieser `switch` ist die einzige Stelle im gesamten Projekt, an der die konkreten Modellklassen genannt werden. Ein drittes Modell einzubauen heißt: eine neue Datei anlegen und hier einen `case` ergänzen - Simulator und Stats bleiben unberührt.

```
returns[i] = terminal[i] / s0 - 1.0;
```

Beachtenswert: Für die Auswertung wird die **einfache** Rendite über den gesamten Zeitraum gebildet, nicht die logarithmische. Innerhalb der Simulation wird logarithmisch gerechnet, weil das numerisch günstig ist - berichtet wird aber die Zahl, die ein Mensch erwartet: plus 8 Prozent heißt aus 100 wurden 108.

Die Zeitmessung mit `System.nanoTime()` umschließt **nur** `sim.run(...)`. Die ausgewiesene Laufzeit ist also die reine Rechenzeit der Simulation, ohne das Sortieren in `Stats` und ohne die Ausgabe. Ein Standardlauf mit 100.000 Pfaden braucht auf diesem Rechner rund 90 Millisekunden.

exportPaths()

```
private static void exportPaths(Simulator sim, double s0, int days, String file)
```

Zweck: Schreibt 20 vollständige Beispielpfade als CSV-Datei, damit die Verläufe in einem Tabellenprogramm gezeichnet werden können.


```
day, path1, path2, path3, ...
0, 100.0000, 100.0000, 100.0000, ...
1, 100.5998, 100.2472, 98.4959, ...
```

Die erste Spalte ist die Tagesnummer, danach folgt je eine Spalte pro Pfad. Dieses Format lässt sich ohne Umbau direkt als Liniendiagramm darstellen.

Der `PrintWriter` steht in einem `try`-mit-Ressourcen-Block und wird daher auch bei einem Fehler zuverlässig geschlossen. Schlägt das Schreiben fehl, wird die Meldung nach `System.err` ausgegeben, ohne das Programm abzubrechen - die Kennzahlen sind zu diesem Zeitpunkt bereits vollständig ausgegeben und sollen nicht wegen eines Dateiproblems verloren gehen.

parseArgs()

```
private static Map<String, String> parseArgs(String[] args)
```

Zweck: Wandelt die Argumentliste in eine Zuordnung von Name zu Wert. Java bringt keine Argumentbibliothek mit, also übernimmt das diese Methode.

```
for (int i = 0; i < args.length; i++) {
    if (!args[i].startsWith("--")) {
        throw new IllegalArgumentException("Unexpected argument: " + args[i]);
    }
    String key = args[i].substring(2);
    if (i + 1 >= args.length || args[i + 1].startsWith("--")) {
        throw new IllegalArgumentException("Missing value for --" + key);
    }
    opts.put(key, args[++i]);
}
```

Die Methode erwartet strikt abwechselnd Name und Wert. Zwei Fehlerfälle werden früh abgefangen: ein Argument ohne `--` davor und ein Name ohne folgenden Wert. Beides führt zu einer klaren Meldung statt zu einem unverständlichen Folgefehler.

Aus dieser Bauweise folgt, dass es **keine** Schalter ohne Wert gibt - jede Option braucht eine Angabe. `--target` und `--export` werden deshalb nicht über einen Wahrheitswert, sondern über `opts.containsKey(...)` erkannt: Ihre bloße Anwesenheit in der Zuordnung schaltet die jeweilige Zusatzausgabe frei.

getD()

```
private static double getD(Map<String, String> opts, String key, double def)
```

Zweck: Holt einen Zahlenwert aus der Zuordnung oder liefert den Standardwert, wenn die Option nicht angegeben wurde. Diese eine Zeile ist der Grund, warum das Programm ganz ohne Argumente lauffähig ist.

```
String v = opts.get(key);
return v == null ? def : Double.parseDouble(v);
```

Auch ganzzahlige Angaben laufen über diese Methode und werden danach umgewandelt, etwa `(int) getD(opts, "days", 252)`. Eine Eingabe wie `--days 252.9` wird deshalb stillschweigend auf 252 abgeschnitten, statt einen Fehler zu melden.

Die Optionen im Überblick

Option	Standard	Bedeutung
<code>--s0</code>	100	Startkurs

Option	Standard	Bedeutung
--mu	0.08	Erwartete Jahresrendite als Dezimalzahl
--sigma	0.20	Jahresvolatilität als Dezimalzahl
--days	252	Zeithorizont in Börsentagen
--paths	100000	Anzahl der simulierten Pfade
--seed	42	Startwert des Zufallsgenerators
--model	gbm	gbm oder jump
--lambda	4	Nur bei jump: Sprünge pro Jahr
--jump-mean	-0.03	Nur bei jump: mittlere Sprunghöhe
--jump-std	0.05	Nur bei jump: Streuung der Sprunghöhe
--target	-	Zusätzlich die Wahrscheinlichkeit für diesen Endkurs ausgeben
--export	-	20 Beispielpfade in diese CSV-Datei schreiben

Teil F - Ergebnisse lesen und prüfen

Die Probe aufs Exempel

Für die GBM gibt es geschlossene Formeln für Mittelwert und Median. Sie eignen sich als Prüfstein: Weicht die Simulation davon ab, stimmt etwas im Code nicht.

Kennzahl	Formel	Erwartet	Simuliert
Mittelwert	$S_0 * e^{(\mu * T)}$	108.33	108.42
Median	$S_0 * e^{((\mu - \sigma^2/2) * T)}$	106.18	106.26
P(Verlust)	aus der Normalverteilung	38.2 %	38.1 %

Alle drei Werte liegen im Rahmen der Monte-Carlo-Streuung. Damit sind die Driftkorrektur, die Skalierung mit der Wurzel der Zeit und die Rückrechnung aus dem Logarithmus gemeinsam bestätigt - drei Stellen, an denen ein Vorzeichen- oder Faktorfehler leicht unbemerkt bliebe.

Solche Prüfungen sind bei Simulationen unverzichtbar. Ein Programm dieser Art stürzt bei einem Denkfehler nicht ab, sondern liefert weiterhin plausibel aussehende Zahlen - nur eben falsche. Wenn ein neues Modell hinzukommt, sollte zuerst ein Fall gerechnet werden, dessen Ergebnis man unabhängig kennt.

Beide Modelle nebeneinander

Kennzahl	GBM	Sprungdiffusion
Mittelwert Endkurs	108.42	108.29
Median Endkurs	106.26	105.72
Verlustwahrscheinlichkeit	38.1 %	40.4 %
VaR 99 %	-33.3 %	-39.3 %
CVaR 99 %	-37.4 %	-44.6 %
Schlimmster Rückgang	57.8 %	71.0 %

Die Tabelle zeigt in einem Bild, was Sprünge bewirken: Der Erwartungswert bleibt praktisch unverändert - dafür sorgt die Kompensation -, aber jede einzelne Risikokennzahl verschlechtert sich deutlich. Wer nur auf die erwartete Rendite schaut, hält beide Welten für gleich.

Grenzen des Modells

Ein Simulator liefert immer Zahlen. Ob sie etwas über die Wirklichkeit aussagen, hängt an den Annahmen - und die sind hier bewusst einfach gehalten:

- **Konstante Volatilität.** In Wirklichkeit tritt Schwankung in Wellen auf: Auf unruhige Tage folgen unruhige Tage. Beide Modelle nehmen σ als über den gesamten Zeitraum unverändert an.
- **Unabhängige Tage.** Jeder Zeitschritt wird unabhängig gezogen. Trendphasen und Gegenbewegungen entstehen im Modell nur zufällig, nicht systematisch.
- **Vorgegebene Drift.** μ ist eine Eingabe, keine Vorhersage. Die Simulation sagt nicht, wie eine Aktie sich entwickelt - sie sagt, welche Bandbreite plausibel ist, wenn die Annahmen zutreffen.
- **Keine Kosten.** Gebühren, Spreads und Steuern kommen nicht vor.
- **Tagesraster.** Ein Rückgang wird nur zu Tagesschlusskursen gemessen; innerhalb eines Tages kann es tiefer gegangen sein, als die Auswertung ausweist.

Die Sprungdiffusion mildert genau eine dieser Schwächen - die zu dünnen Ränder - und lässt die übrigen unberührt. Deshalb ist sie kein Ersatz für die GBM, sondern ein Werkzeug für eine bestimmte Frage: Wie verändert sich das Risikobild, wenn Schocks möglich sind? Für diese Frage lohnt der Vergleich beider Läufe.

Aufrufe zum Ausprobieren

```
cd /Users/phi/VSCode/monteCarloSimulation

# Standardlauf: GBM, 1 Jahr, 100.000 Pfade
java MonteCarloSim.java

# Kurzer Horizont bei hoher Volatilität
java MonteCarloSim.java --sigma 0.35 --days 63

# Sprungmodell mit häufigeren, stärkeren Schocks
java MonteCarloSim.java --model jump --lambda 6 --jump-mean -0.04

# Wahrscheinlichkeit für einen Endkurs von mindestens 120
java MonteCarloSim.java --target 120

# 20 Beispielpfade für ein Diagramm exportieren
java MonteCarloSim.java --export paths.csv
```

Ein lohnender erster Versuch ist der direkte Vergleich: einmal mit `--model gbm` und einmal mit `--model jump` bei sonst gleichen Vorgaben. Der Mittelwert bleibt nahezu gleich, während CVaR und schlimmster Rückgang sichtbar auseinanderlaufen - das ist der gesamte Unterschied zwischen den beiden Modellwelten in zwei Zeilen Ausgabe.